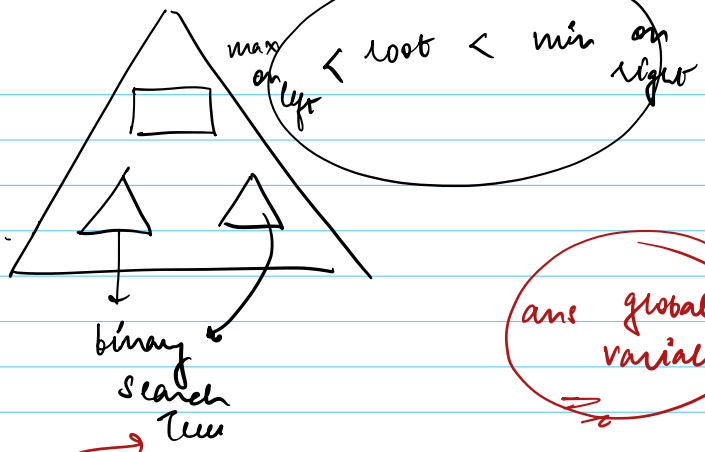
Largest BST subtree

idea



ans global variable

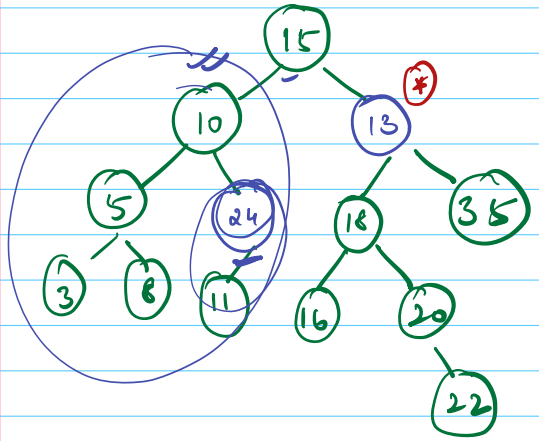
< size, bool → isBST >

if root == NULL

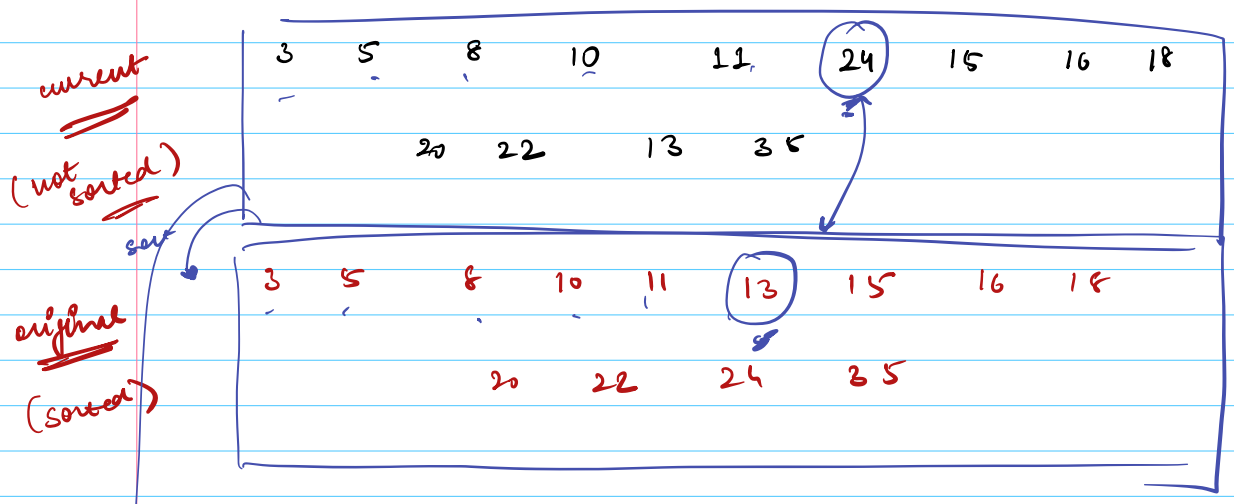
< 0, True >

Q = I had a BST (distinct values) . I swapped 2 nodes and gave that tree to you.

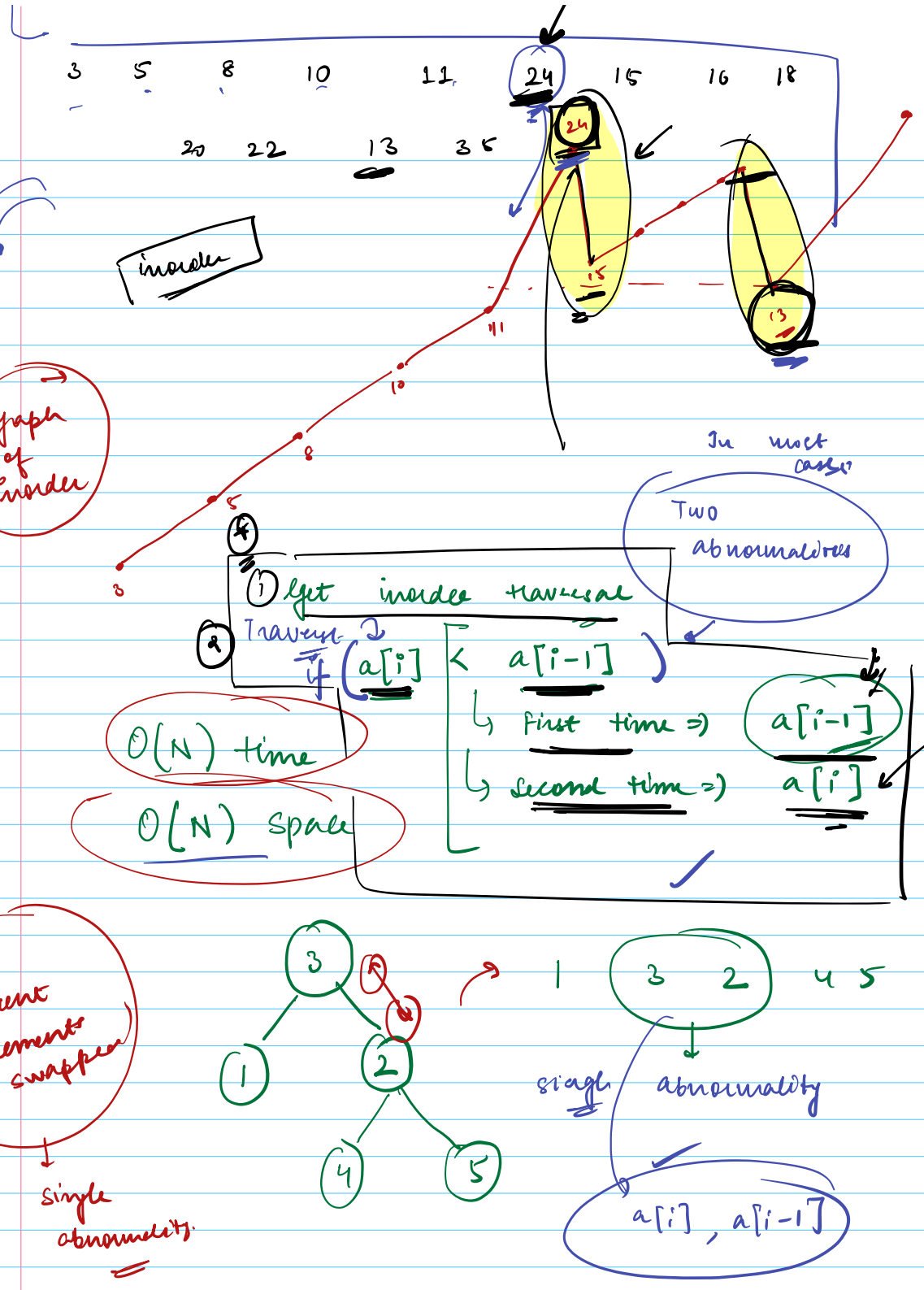
Find those 2 swapped nodes



Range idea might not be able to single out on the swapped nodes.



compare inorder traversals , first mismatch → which elements were swapped


3 5 8 10 11 24 15 16 18
20 22 13 35
inorder
graph of inorder
In most cases
Two abnormalities
1) Get inorder traversal
2) Traverse
if (a[i] < a[i-1])
First time => a[i-1]
Second time => a[i]
O(N) time
O(N) space
If adjacent elements swapped
single abnormality
1 3 2 4 5
single abnormality
a[i], a[i-1]

O(n) space

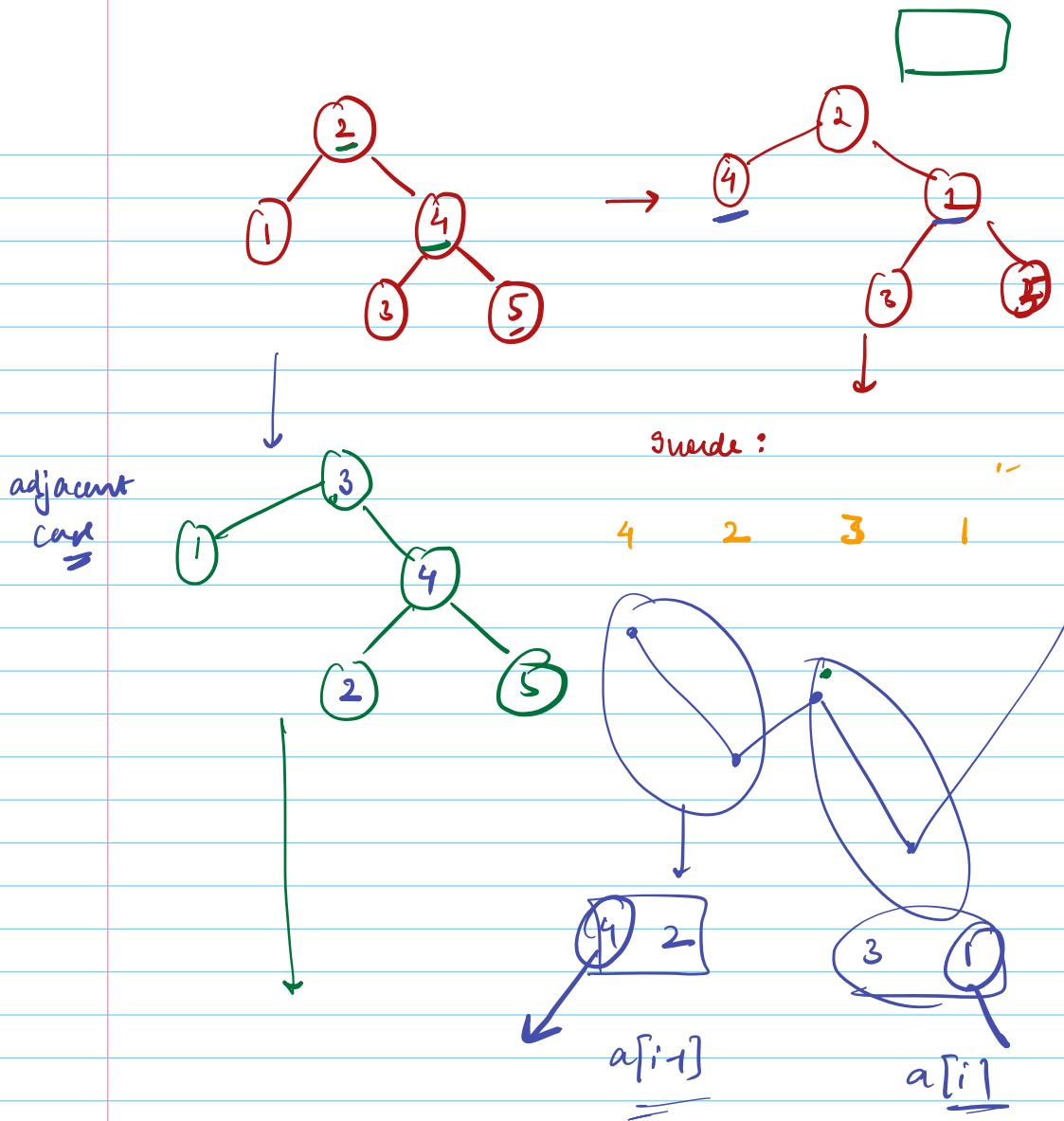
previous node of inorder traversal

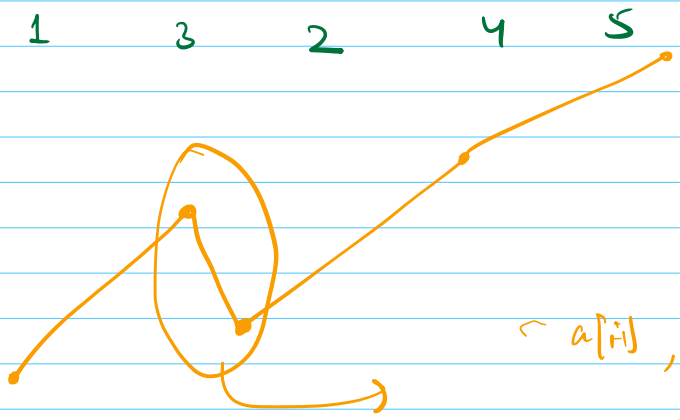
prev = NULL

first = NULL , second = NULL

```
find ( Node root )

if ( root == NULL )
    return ;

find ( root.left )

if ( prev != NULL
     and
     prev.data > root.data ){

    if ( first == NULL )
        first = prev , second = root

    else
        second = root

prev = root.

find ( root.right )
```

visit curr node


adjacent case
Swede :
4 2 3 1 5
a[i-1]
a[i]


1 3 2 4 5
a[i-1], a[i]

space

O(H)

Inorder

prev = NULL

Inorder (Node curr)

if (curr == NULL) return

inorder (curr.left)

if curr < prev

visit current node

prev = curr

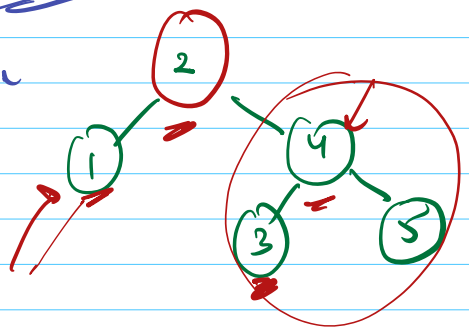
if (first == NULL)

first = prev

second = curr

inorder (curr.right)

if second = curr



1 visited => prev = NULL

2

3

BBST ⇒ $\log N$

Search an Element in BST

↓

TC : O (Height)

Skewed Tree

Can you get the path to the element

In a BT ⇒ search

bool search (Node root, int K)

O(N)

Extra Requirement

Path to the element from root

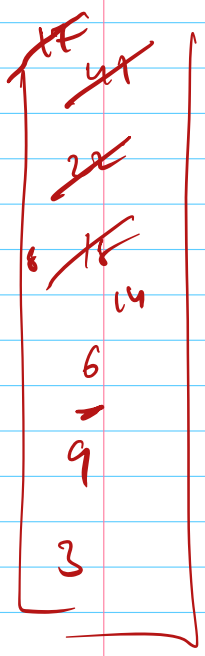
if (search (left))
return True
if root = K

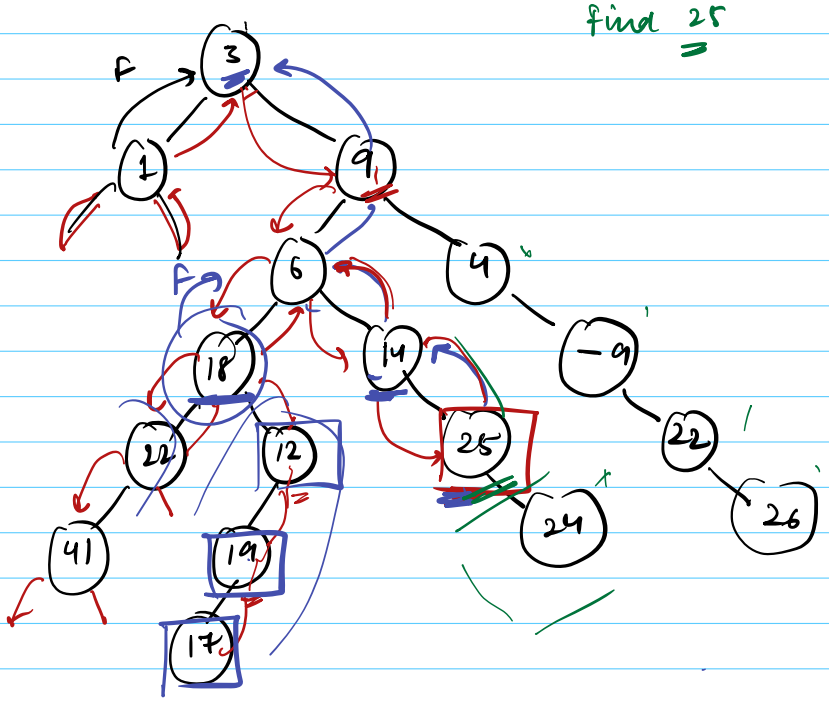
if (search (right))
return true

return search (left) || search (right)

Red lines deficit searching

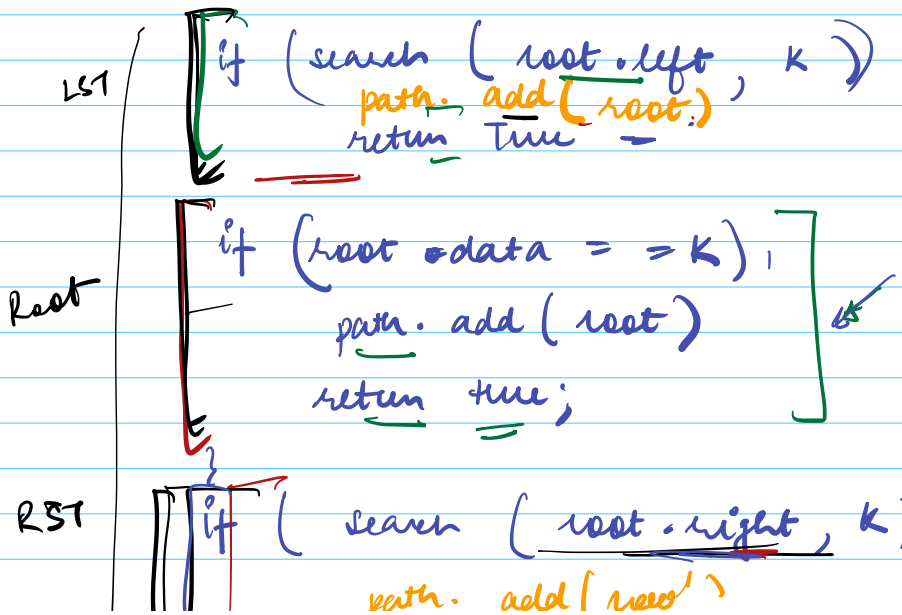
find 25





path = [3, 14]

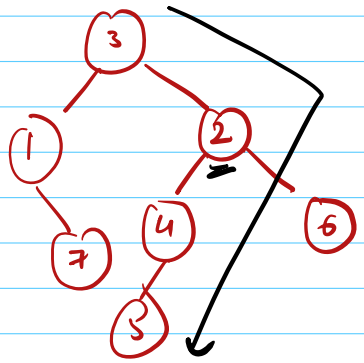
```
bool search ( Node root , int K )
    if (root == NULL ) return False
```

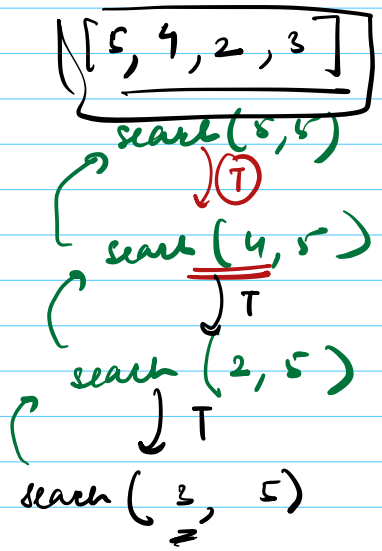


for value
path.add (root.data)

return True

return False

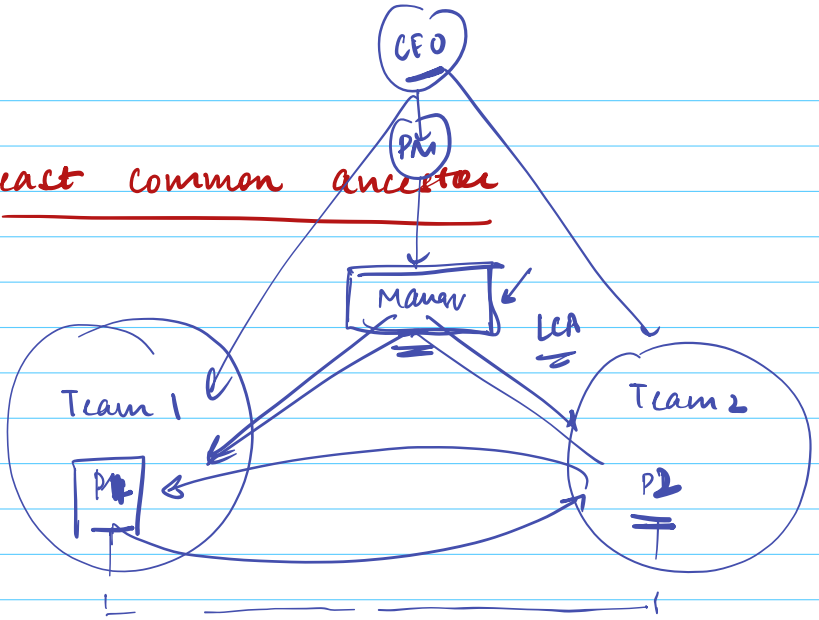


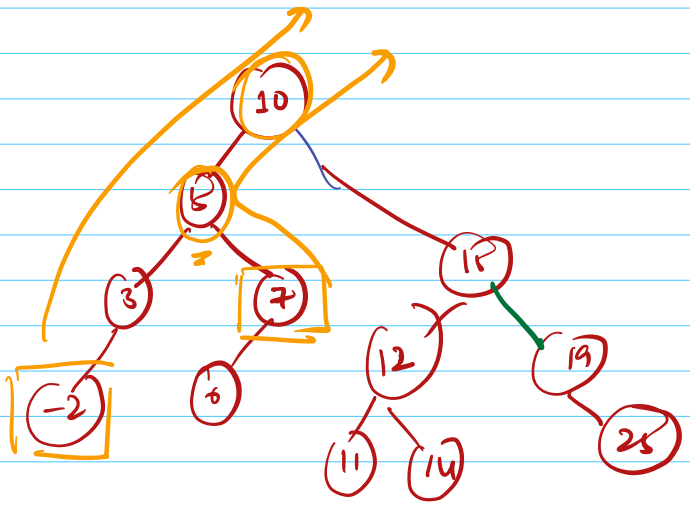


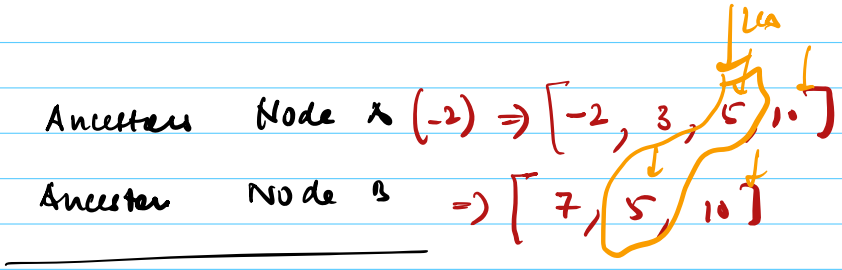
Reverse the path in the end

LCA

Least common ancestor

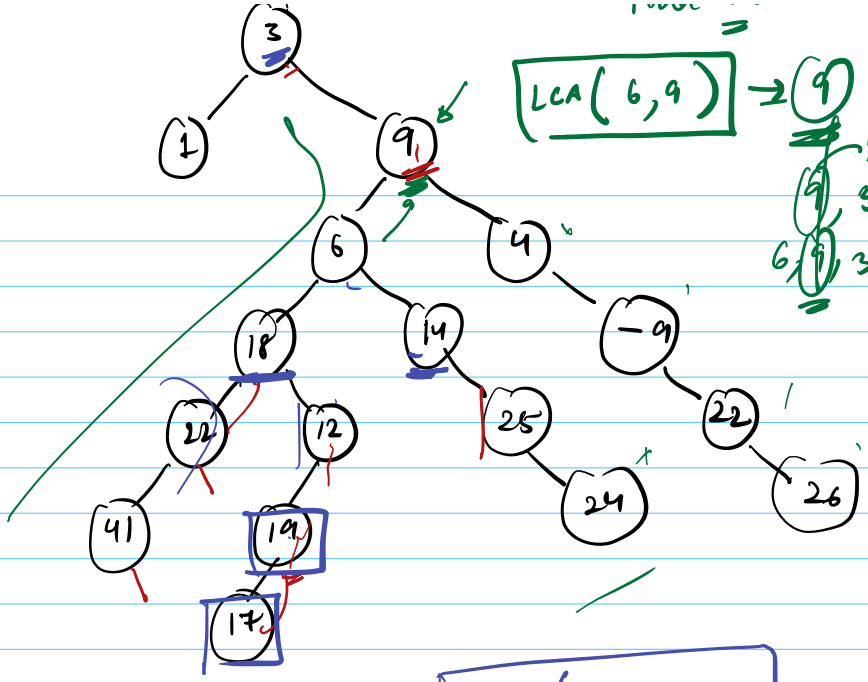






Most younger ancestor common to both => LCA

find 25



LCA(6, 9) → 9

LCA(6, -9) → 9

LCA(17, 26) ⇒ 9

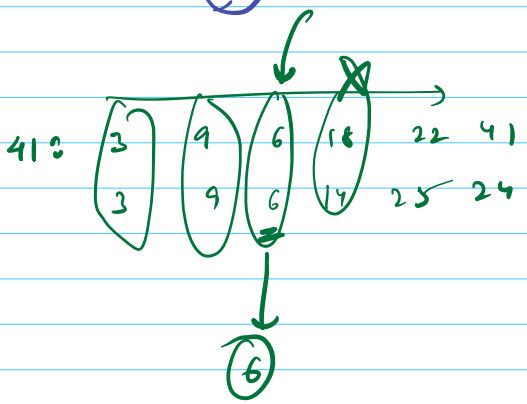
LCA(41, 24) ⇒ 6

LCA(1, 25) ⇒ 3

Algo ⇒

Path (A)

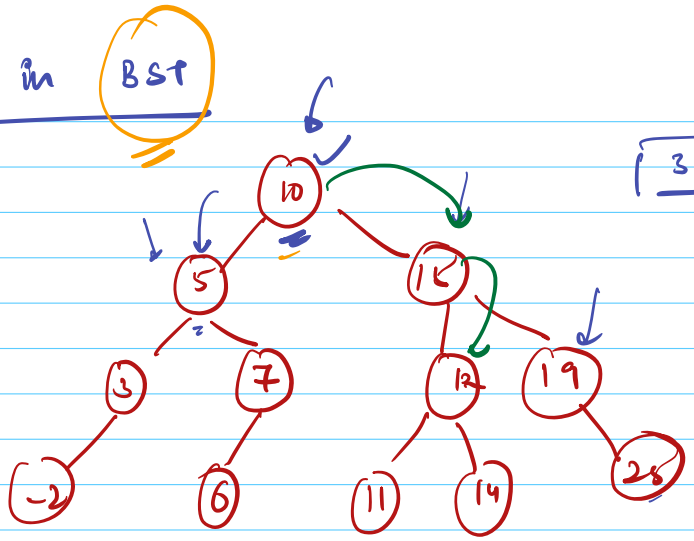
Path (B)



SC: O(N) → O(H)

TC: O(N)

(*) LCA in BST



3 and 6

LCA (19, 25) → 19

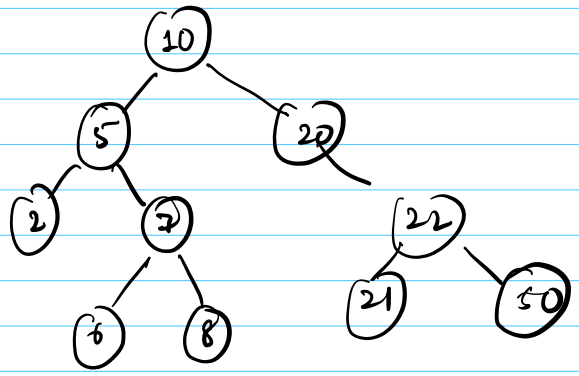
LCA (11, 14)

TC = O(height)

SC = O(1)

BBST → T(O Log(N))

Q Find K^{th} smallest element in BST



BF ⇒ Inorder ⇒ Inorder [K-1] → K^{th} smallest element!

SC → O(N)

save space

maintain a counter

TC = O(N)

SC = O(H) → Recursion

V.V.V Hard

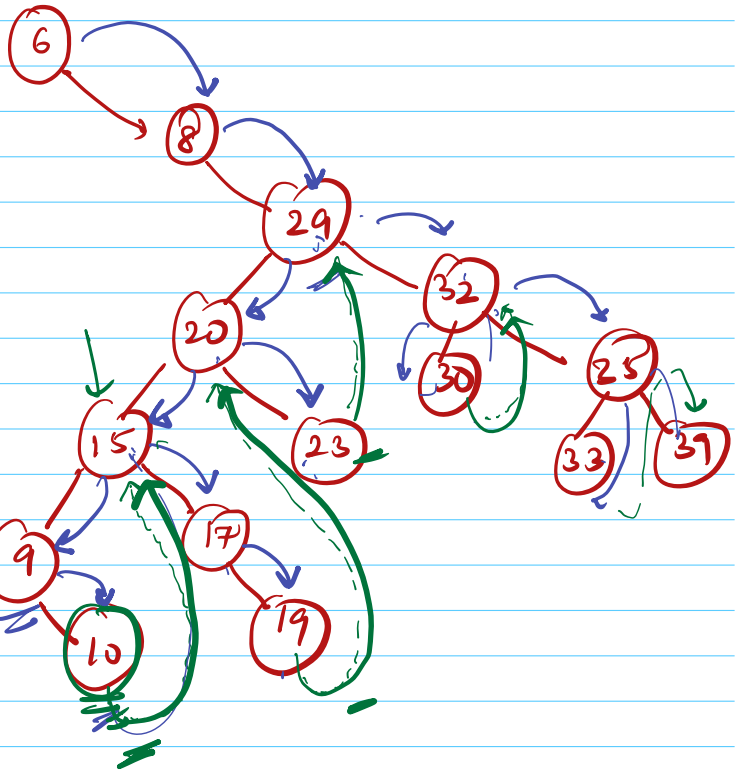
MORRIS INORDER TRAVERSAL

O(H)

Inorder Traversal → O(1) space

COST => It modifies structure of tree

Inorder predecessor



6 8 9 10 15 17 19 20 23 29

30 32 33 35 39

4 dec

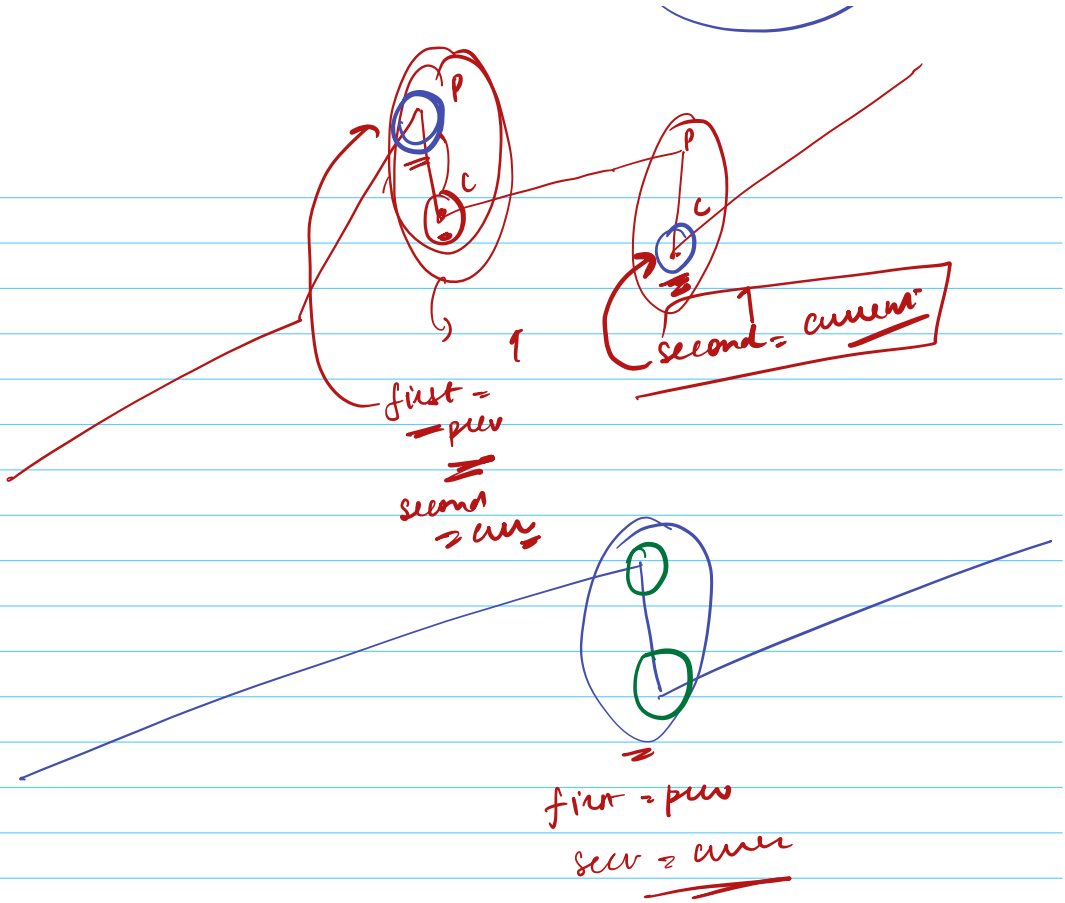
Doubt session

4PM → 6PM

✓ Hands on

✓ Morris Inorder Traversal

✓ Doubts in LL, Stack Queue


p
c
first = prev
second = curr
p
c
second = current
first = prev
sec = curr